

Bancos de Dados de Grafos — guia de estudo

Neo4j · ICIJ Offshore Leaks · comparação SQL vs Cypher

BD2 · UFPR · Tema 9 · 17/06/2026 · slides: ocoiel.github.io/ufpr-bd2-graph-databases

Como apresentar. Alvo: **12–15 min** de fala + demo ao vivo. Controles do deck: ← → navegam · N abre o roteiro de cada slide · F tela cheia.

CHECKLIST ANTES DE COMEÇAR

- `docker compose ps` → **bd2-neo4j** e **bd2-postgres** "healthy"
- `cd app && pnpm dev` → `localhost:3000`
- Abas abertas: o deck · `localhost:3000` (app) · `localhost:7474` (Neo4j Browser, opcional)
- ⚠ Postgres do projeto roda na porta **5433** (a 5432 já tem um Postgres nativo na máquina)

AS 3 IDEIAS QUE SUSTENTAM TUDO

- **O relacionamento é cidadão de primeira classe.** No grafo ele é **armazenado** como aresta; no relacional ele é **recalculado com JOIN** a cada consulta.
- **Index-free adjacency.** Cada nó aponta direto pros vizinhos → seguir uma aresta é **O(1) local**, independente do tamanho do banco. Por isso o multi-hop não degrada.
- **A tradução é trivial; o desempenho, não.** SQL→Cypher é fácil de escrever, mas o tempo diverge de empate (busca rasa) a centenas de vezes (multi-hop), até onde o SQL não termina ou não consegue expressar.

ROTEIRO SLIDE A SLIDE

01 Abertura

~30s

FALE

"Nosso tema é **banco de dados de grafos**, com o Neo4j. Vamos do simples — o que é, pra que serve, um 'olá mundo' — até uma demo investigativa real com a base dos **Panama Papers**, comparando lado a lado com SQL."

Diga os nomes da dupla e o tema (Tema 9).

02 O que é um banco de grafos

~1min

NO SLIDE

- **Nó** = uma entidade (pessoa, empresa)
- **Relacionamento** = ligação com **direção** e **tipo**
- **Propriedades** = chave-valor, em nós e em relacionamentos (modelo "property graph")

FALE

"Relacional guarda **tabelas e linhas**. Grafo guarda **nós e relacionamentos** — como você desenharia num quadro branco. A frase pra guardar: o relacionamento é **armazenado**, não recalculado com JOIN."

03 Pra que serve

~1min

NO SLIDE

- Redes sociais, recomendação, detecção de fraude, logística, knowledge graphs pra IA, investigação financeira

FALE

"Quando a pergunta é sobre o **caminho entre as coisas**, e não sobre as coisas isoladas, o grafo brilha. E é honesto dizer: não é bala de prata — pra relatório tabular, soma e média, o relacional continua melhor."

04 Relacional vs. Grafo

~1min30

RELACIONAL • 3 TABELAS + JOIN

```
SELECT e.nome FROM pessoas p
JOIN trabalha_em t ON t.pessoa_id=p.id
JOIN empresas e ON e.id=t.empresa_id
WHERE p.nome='Ana';
```

GRAFO • O PADRÃO É O DESENHO

```
MATCH (p:Pessoa {nome:'Ana'})
-[:TRABALHA_EM]→(e:Empresa)
RETURN e.nome;
```

FALE

"Mesma informação, dois modelos. O ponto que importa: cada 'pulo' a mais — amigo do amigo do amigo — no relacional é **mais um JOIN**; no grafo é só estender o desenho. É daí que vem a diferença de desempenho."

05 "Olá, mundo" em Cypher

~1min30

NO SLIDE

```
// 1 · criar
CREATE (ana:Pessoa {nome:'Ana'}),
      (acme:Empresa {nome:'ACME'}),
      (ana)-[:TRABALHA_EM {desde:2020}]>(acme);
// 2 · consultar
MATCH (p:Pessoa)-[:TRABALHA_EM]>(e:Empresa) RETURN p.nome, e.nome;
```

FALE

"Cypher é a 'SQL dos grafos'. A sintaxe é **ASCII-art**: parênteses são nós, setas são relacionamentos. `CREATE` grava; `MATCH ... RETURN` casa o padrão e devolve. Você descreve o desenho que procura — o banco acha."

Opcional ao vivo (Neo4j Browser, `localhost:7474`): cole o CREATE, depois o MATCH, mostre o nó e a aresta desenhados.

06 O dataset — ICIJ Offshore Leaks

~1min

2,0 mi nós

3,3 mi relações

814k empresas

771k pessoas

27k intermediários

402k endereços

FALE

"Base real por trás dos Panama, Paradise e Pandora Papers. **2 milhões de nós, 3,3 milhões de relações**, rodando local no Docker. O pulo do gato: o **mesmo dataset** está espelhado no Neo4j e no PostgreSQL — pra fazer a mesma pergunta nos dois e medir."

07-08 Demo: busca por nome (query simples)

~1min · ao vivo

PASSOS NO APP (LOCALHOST:3000)

- Buscar "**Putin**" → mostra Alexander Putin, Igor Putin...
- Apontar o painel inferior: o Cypher é uma linha (`MATCH ... CONTAINS`)
- Clicar num resultado → abre a vizinhança no grafo

FALE

"Essa é a operação **rasa**: aqui SQL e Cypher **empatam**, os dois usam índice. Guardem esse empate — a partir daqui o grafo abre vantagem."

09 Multi-hop — o pulo do gato

~2min · ao vivo

A PERGUNTA

1 hop: empresas que a pessoa controla. **2 hops com volta:** "quem mais usou o mesmo intermediário?" — o padrão investigativo clássico.

```
// Cypher: um padrão
MATCH (o:Officer)-[:officer_of]→(e1)←[:intermediary_of]-(i)
      -[:intermediary_of]→(e2)←[:officer_of]-(outro:Officer)
RETURN DISTINCT outro.name;
// O mesmo em SQL: 4 JOINS sobre a tabela de relações
```

FALE

"Mesma pergunta. No Cypher é **um padrão**; o mesmo em SQL já são **4 JOINS**. Um lado é um desenho, o outro é um quebra-cabeça de junções."

10 Caminho mais curto entre 2 pessoas

~1min30 · ao vivo

```
MATCH p = shortestPath( (a)-[*..6]-(b) ) RETURN p; // Cypher: 1 linha
// SQL: WITH RECURSIVE ... CTE que reexpande a fronteira a cada nível
```

FALE

"O caso onde o SQL sofre: a profundidade é **desconhecida**. No Cypher é `shortestPath`, uma linha. No SQL é um CTE recursivo cuja fronteira explode. A partir de um nó bem conectado, o SQL **estoura o timeout já em 3 hops**; o Cypher responde em ~6ms em qualquer profundidade."

No app: aba Algoritmos → caminho mais curto entre dois nós.

11 Benchmark — SQL vs Cypher medido

~2min · slide-chave

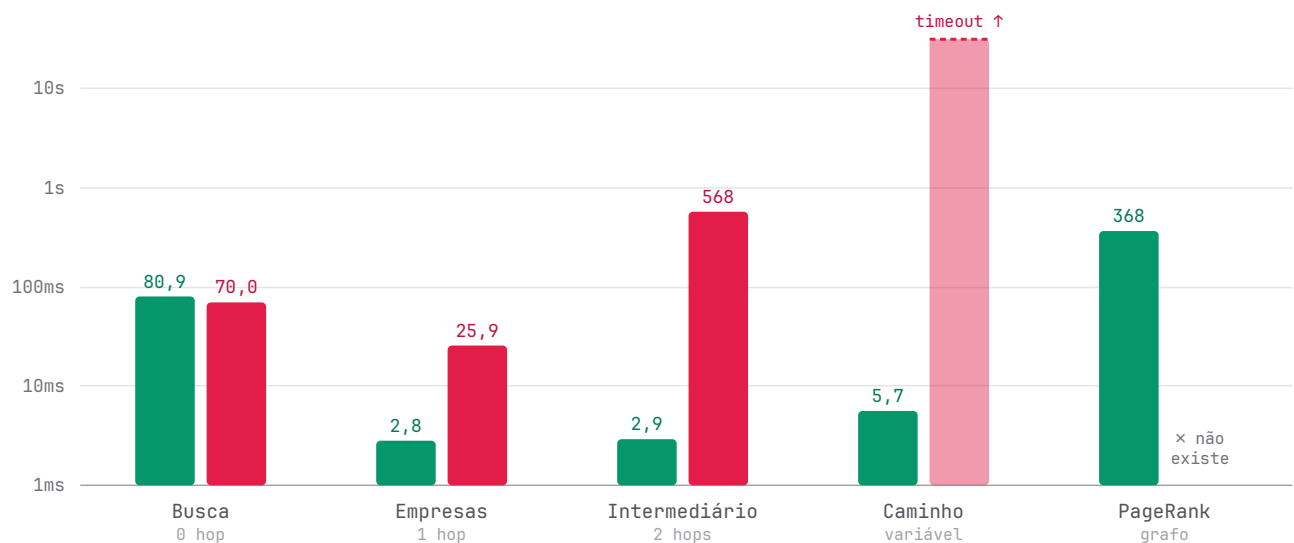
PERGUNTA	HOPS	CYPHER	SQL	VENCEDOR
Buscar por nome	0	80,9 ms	70,0 ms	empate
Empresas controladas	1	2,8 ms	25,9 ms	Cypher 9×
Mesmo intermediário	2	2,9 ms	567,6 ms	Cypher 193×
Caminho mais curto	variável	5,7 ms	timeout >20s	Cypher ✓
PageRank	grafo	367,9 ms	não existe	Cypher ✓

FALE

"Não é achismo — medimos. Mesmo dataset, mesma pergunta, aquecimento + 5 repetições (`pnpm bench`). Quanto mais a pergunta depende de relacionamentos, mais o grafo ganha: de empate a ordens de grandeza."

11b O mesmo, em gráfico (escala log)

~1min



Escala logarítmica (cada degrau = 10×). Verde = Cypher, vermelho = SQL. A barra do caminho fura o teto (timeout); PageRank não tem barra vermelha (SQL não expressa). A distância verde↔vermelho é a vantagem do grafo — e cresce da esquerda pra direita.

12 "E com 3, 4, 5 hops?"

~1min30

NÓ ESPARSO (~4 CONEXÕES)			NÓ DENSO / HUB (CASO REAL)		
HOPS	CYPHER	SQL	HOPS	CYPHER	SQL
3	2,9	0,5 ms	2	7,4	182 ms
4	1,9	0,8 ms	3	6,8	timeout
5	2,0	3,0 ms	4	8,0	timeout
6	2,0	25,7 ms	5	6,8	timeout

FALE

"Escala com a profundidade **depende da densidade do nó de partida**. Nó esparsos: SQL acompanha até 4–5 hops. Nó denso (o caso interessante): SQL estoura já em 3. Nos dois, o Cypher fica **~constante**. A chave: o SQL não escala com a profundidade — escala com **densidade × profundidade**."

13 Algoritmos de grafo (GDS)

~1min30 · ao vivo

- **PageRank** — o algoritmo do Google: quem são os nós mais **influentes**
- **Louvain** — detecção de **comunidades** (clusters densamente ligados)

FALE

"O grafo não só consulta, ele **analisa a estrutura**, via a biblioteca GDS. Isso, em SQL puro, **não existe** — teria que exportar pra Python ou Spark. No Neo4j é uma chamada."

💡 **Dica:** rode os algoritmos sobre a vizinhança de uma **pessoa comum** (resultado da busca), não sobre um intermediário gigante — o subgrafo de um hub é enorme e demora.

14 Quando usar grafo (e quando não)

~1min

GRAFO BRILHA

- Relacionamentos profundos e variáveis
- Caminhos, padrões de conexão
- Recomendação, fraude, knowledge graph

RELACIONAL GANHA

- Dados tabulares, agregações em colunas
- Relatórios, BI, somas e médias
- Transações simples e bem normalizadas

FALE

"Grafo não substitui o relacional. Na prática usa-se os dois — persistência poliglota, cada um no que é bom. Foi o que fizemos: espelhamos o mesmo dado nos dois."

15 Conclusão

~45s

"A **tradução** de SQL pra Cypher é trivial; o **desempenho**, não. Vai do empate na busca rasa às centenas de vezes nos multi-hop, até onde o SQL não termina nem consegue expressar. Quanto mais a pergunta é sobre conexões, mais o grafo ganha. Stack: Neo4j 5.26 + GDS + APOC, PostgreSQL 17, Next.js — tudo local em Docker."

CONCEITOS & CONTEXTO (PRA ENTENDER, NÃO DECORAR)

abc O que é Cypher, exatamente

- A linguagem de consulta do Neo4j — o que o SQL é pro relacional. Três marcas:
- **Declarativa** — você diz o *quê* quer, não o *como* buscar (igual SQL nesse ponto).
 - **ASCII-art** — você desenha o padrão: `( )` é um nó, `-[ ]→` é uma relação com direção. O código parece o diagrama.
 - **Padrão ISO** — desde 2024 virou o **GQL** (Graph Query Language), aprovado pela ISO — mesmo comitê do SQL. Versão aberta: openCypher.

CYPHER	SQL EQUIVALENTE	O QUE FAZ
CREATE	INSERT	cria nós/relações
MATCH ... RETURN	SELECT ... FROM	acha um padrão e devolve
WHERE	WHERE	filtra
MERGE	INSERT ... ON CONFLICT	cria se não existir

"Cypher é a 'SQL dos grafos'. A sintaxe é ASCII-art: parênteses são nós, setas são relacionamentos. E desde 2024 virou padrão ISO, o GQL — o mesmo comitê do SQL."

🌐 Quem usa grafo de verdade

Google	PageRank ranqueia páginas pela estrutura de links (+ Knowledge Graph)
Meta	Social Graph — "amigos de amigos"
LinkedIn	conexões de 1º/2º/3º grau = distância em hops
Netflix/Amazon	recomendação usuário→produto→usuário
Bancos	fraude: contas "diferentes" que dividem telefone/device
ICIJ	Panama/Pandora Papers — o nosso dataset

Abertura matadora: "se você já usou o Google, já usou um algoritmo de grafo — o mesmo PageRank que vou rodar na demo".

Panama · Paradise · Pandora Papers

Três grandes **vazamentos** sobre **offshores** (empresas em paraísos fiscais, dono escondido, usadas pra esconder/sonegar/lavar). Investigados pelo **ICIJ** (Consórcio Internacional de Jornalistas Investigativos). O **intermediário** — escritório que cria as offshores — é a peça central: "quais clientes diferentes usaram o mesmo intermediário?".

VAZAMENTO	ANO	FONTE	DOCS
Panama	2016	Mossack Fonseca (Panamá)	11,5 mi
Paradise	2017	Appleby (Bermudas)	13,4 mi
Pandora	2021	14 escritórios offshore	~12 mi

Vêm de fontes anônimas. Renderam o **Pulitzer 2017** e derrubaram o PM da Islândia. **O ponto:** o ICIJ usou Neo4j de verdade e publicou a base como grafo (ODbL) — é o nosso dataset. Não é exemplo de brinquedo.

Index-free adjacency — o conceito que explica tudo

Cada nó guarda, fisicamente, **ponteiros diretos pros vizinhos**. Seguir uma aresta é **O(1) local** — ter 2 milhões ou 2 bilhões de nós não muda o custo do pulo. Por isso o multi-hop não degrada. No SQL, cada hop é um JOIN que varre/indexa a tabela de relações inteira de novo.

Analogia da agenda. Grafo = o telefone dos amigos salvo no seu celular (instantâneo). JOIN do SQL = toda vez, procurar o nome numa **lista telefônica da cidade inteira**. Com 1 amigo tanto faz; com "amigo do amigo do amigo", a lista telefônica te mata.

Por que o caminho variável quebra o SQL: a fronteira do CTE recursivo cresce exponencialmente → problema **algorítmico**, não de índice. Índice não resolve. (Decore isso.)

PERGUNTAS PROVÁVEIS (Q&A)

Por que o grafo é mais rápido no multi-hop?

Index-free adjacency: cada nó guarda ponteiros diretos pros vizinhos. Seguir uma aresta é O(1) local, independente do tamanho total do banco. No SQL, cada hop é um JOIN que varre/indexa a tabela de relações inteira de novo.

Então é sempre melhor usar grafo?

Não. Pra dado tabular, agregação e relatório, o relacional é mais simples e eficiente. Grafo ganha quando o **relacionamento** é o protagonista.

O Postgres não ficaria mais rápido com mais índices/tuning?

Ajuda nos hops rasos. Mas o problema do caminho de profundidade variável é **algorítmico** (a fronteira do CTE cresce exponencialmente) — índice não resolve. E PageRank/Louvain não têm equivalente declarativo.

O que é GDS e APOC?

GDS = Graph Data Science, a lib de algoritmos (PageRank, Louvain, caminhos). **APOC** = biblioteca de utilitários/procedimentos do Neo4j.

Os dados são reais? Como garantem que é a mesma comparação?

Sim — ICIJ Offshore Leaks, licença ODbL, 2M nós / 3,3M relações. O Neo4j é a fonte; um ETL (`pnpm mirror:pg`) espelha **exatamente** os mesmos nós e relações pro schema relacional. O `id` do Neo4j vira `node_id` no Postgres — mesma chave dos dois lados.

O que é Cypher?

A linguagem de consulta declarativa do Neo4j (hoje padronizada como GQL/openCypher). Descreve **padrões** de nós e relacionamentos em sintaxe ASCII-art.

SE ALGO QUEBRAR NA HORA

PROBLEMA	AÇÃO
App não conecta	<code>docker compose ps</code> — Neo4j "healthy"? Se reiniciou, espera ~60s (recarrega plugins).
Busca vazia	Tenta <code>Putin</code> , <code>Mossack</code> , <code>Limited</code> .
Algoritmo travando	Você rodou num hub gigante. Rode numa pessoa comum (poucos vizinhos).
Postgres "role does not exist"	Está batendo no Postgres nativo (5432). Confirme <code>POSTGRES_PORT=5433</code> no <code>.env</code> e <code>app/.env.local</code> .
Reprovar os números ao vivo	<code>cd app && pnpm bench</code> e <code>node --env-file=../.env scripts/hop-sweep.mjs</code>